

中国石油大学（北京）

2025—2026 学年春季学期

《人工智能基础》结课报告

报告名称：大模型语言幻觉检测与缓解技术研究

组员姓名 叶浚宇，王晨枫

专业名称 人工智能

指导教师 崔鸿雁

班级	学号	姓名	现状	表达	写作	结果	逻辑	得分
人工智能 24-2 班	2024015937	王晨枫						
人工智能 24-1 班	2024015912	叶浚宇						

中国石油大学（北京）克拉玛依校区

2026 年 6 月 15 日

评分标准

分项	评价细则	等级细则	分值区间
现状	是否有近 3 年文献资料，能够反应现状(形式一)？对现状是否有自己独特的归纳和见解(形式二)？	A、内容详实，符合主题，论述深刻。	16-20
		B、内容一般，基本符合主题，论述不够深刻。	6-15
		C、内容空泛，不符合主题，论述浅显。	0-5
表达	报告表达是否逻辑清晰？让非专业技术人员容易理解。	A、语言准确，文字流畅通顺。	16-20
		B、语言基本准确，文字尚通畅。	6-15
		C、语言不通畅，有错别字	0-5
写作	写作规范：字号小四，缩进和小标题和标号合理，无错别字，排版合理、层次分明。	A、论文写作层次分明，题目切合要求，摘要、关键词及参考文献明确。	16-20
		B、论文层次一般，题目基本切合要求，摘要、关键词及参考文献基本明确。	6-15
		C、论文层次混乱，题目不切合要求，摘要、关键词及参考文献不明确。	0-5
结果	是否有有深度、有价值、全面的结论、归纳？	A、论文具有前瞻性，有较多的创新见解	16-20
		B、论文具有一定前瞻性，有少数的创新见解	6-15
		C、论文缺乏前瞻性，几乎没有创新见解	0-5
逻辑	代码和数据集完整？（理论算法研究类/创新实验设计）；有理有据，有按国内外发展时间线的分析结论、逻辑层层递进（原理阐述/综述类）。	A、逻辑清晰、完整、分析透彻、有自己的总结和见解。	16-20
		B、逻辑较清楚、有分析总结	6-15
		C、缺乏逻辑、缺乏总结分析	0-5
总分	总分 100		

摘 要

大语言模型能够生成流畅自然的文本，但在事实问答、长文本生成、专业知识解释和检索增强生成等场景中，仍可能产生“看起来合理、实际不真实或无法由证据支持”的幻觉内容。幻觉问题直接影响人工智能系统的可信性、安全性与可落地性，也是当前大模型从演示系统走向真实应用时必须面对的基础问题。本文围绕大语言模型幻觉检测与缓解技术展开研究，首先梳理幻觉的定义、分类与产生原因，说明事实性幻觉、忠实性幻觉和时效性幻觉在风险来源上的差异；其次总结近三年代表性研究，包括 SelfCheckGPT[2]、HaluEval[3]、FActScore[4]、RAGAS[5]、Chain-of-Verification[6]和 FreshQA[7]等工作，归纳当前研究从整体判断走向原子事实、证据支持和系统评估的趋势；然后从原子事实抽取、证据检索、一致性判断、检索增强生成和验证式生成五个角度分析关键技术原理。为避免停留在综述层面，本文进一步基于开源代码完成 100 条问题的可复现实验，对基础大模型、RAG、RAG+验证和 RAG+CoVe 四类方案进行比较。真实运行结果显示，四种方法的幻觉率分别为 36.0%、30.5%、17.2%和 14.3%；验证与拒答机制能够降低幻觉，但也会使答案覆盖率从 100%下降到 64%和 14%。实验表明，低幻觉率并不自动等于高可用性，可信问答系统需要同时报告正确率、幻觉率、覆盖率和时延。本文认为，未来可信大模型系统应从“单次生成”转向“检索—生成—验证—拒答”的闭环架构，在有证据时回答，在证据不足时说明不确定性，从而兼顾安全性与实用性。

关键词：大语言模型；幻觉检测；检索增强生成；事实一致性；可信人工智能

目 录

一、 介绍	7
1.1 研究背景与选题意义	7
1.2 本文主要贡献与创新点	8
二、 研究现状	10
2.1 大语言模型幻觉的定义与分类	10
2.2 近三年代表性研究进展	11
三、 存在的科学问题与关键问题	12
四、 研究内容与技术原理	14
4.1 实验架构	14
4.2 幻觉检测任务建模	16
4.3 基于采样一致性的检测方法	16
4.4 基于原子事实与证据检索的检测方法	17
4.5 基于 RAG 与验证式生成的缓解方法	18
五、 研究方案与实验设计	20
5.1 数据集与评价指标	20
5.1.1 测试集（100 条事实型问答样本）	20
5.1.2 知识库（101 份知识材料，证据 + 干扰文档）	22
5.1.3 数据集设计的实验价值	23

5.1.4 评价指标设计	24
5.2 对比方法与实验流程	25
5.2 .1 Base LLM 基础大模型	25
5.2 .2 RAG 检索增强生成	25
5.2 .3 RAG + Verify 事实验证	26
5.2 .4 RAG + CoVe 链式验证	26
5.3 实验结果与分析	28
5.3.1 基础数据分析	28
5.3 .2 检索增强机制能够降低幻觉，但无法完全解决事实错误	30
5.3 .3 验证机制能够进一步降低幻觉，但会降低答案覆盖率	31
5.3 .4 CoVe 方法降低幻觉能力最强，但存在过度拒答问题	31
5.3 .5 综合分析：可信问答系统需要多指标协同优化	32
5.4 代码实现	32
5.5 典型样例分析	36
六、结论与收获	38
6.1 结论	38
6.2 团队技术分工与个人收获	40
参考文献	41
附录代码	43

一、介绍

1.1 研究背景与选题意义

大语言模型（Large Language Models, LLMs）通过在大规模语料上进行预训练，学习了语言规律、知识关联和上下文推理能力。在问答、写作、代码生成和智能体等应用中，大模型已经成为人工智能系统的重要基础组件。然而，大模型并不直接存储结构化事实，也不会天然区分“语言上合理”和“事实可靠”。因此，它可能生成与现实世界、给定材料或用户问题相矛盾的内容，这类现象通常被称为幻觉。

幻觉问题之所以重要，是因为它具有较强的隐蔽性。与普通错误不同，大模型的幻觉往往语气肯定、表达完整、逻辑看似连贯，用户如果缺乏专业背景，很难及时发现错误。例如，在医学、法律、金融、论文写作和工程决策场景中，错误答案可能造成严重后果。因此，如何检测和缓解幻觉，是人工智能基础课程中“知识表示、推理、自然语言处理、机器学习与可信 AI”多个知识点交叉形成的研究问题。

本文选择“基于检索与验证的大语言模型幻觉检测与缓解技术”作为研究点，原因有三：第一，该方向与当前大模型应用密切相关，具有现实意义；第二，该问题既包含模型原理，也包含工程系统设计，适合作为课程报告展开；第三，近三年该方向出现了大量代表性工作，便于从研究现状、关键问题、技术方案和实验结果四个层面进行系统分析。

从课程研究角度看，该选题能够把人工智能基础中的多个知识点连接起来：自然语言处理解释了模型如何理解和生成文本，信息检索解释了外部知识如何进入模型上下文，机器学习评价指标解释了系统效果如何量化，可信 AI 则要求模型在不确定时具备风

险意识。相比单纯介绍某一种模型，幻觉检测与缓解更能体现“模型能力、数据质量、算法流程和工程评估”之间的综合关系。

从应用角度看，幻觉并不只是答案错误那么简单。用户往往会根据模型表达的流畅程度判断可信度，而大模型恰恰能够把错误内容组织得非常完整。因此，缓解幻觉不能只依赖提示词提醒模型“不要胡说”，还需要在系统层面加入检索、证据筛选、事实验证、拒答和日志复评等机制，使生成过程受到外部约束。本文后续实验也围绕这一思路展开。

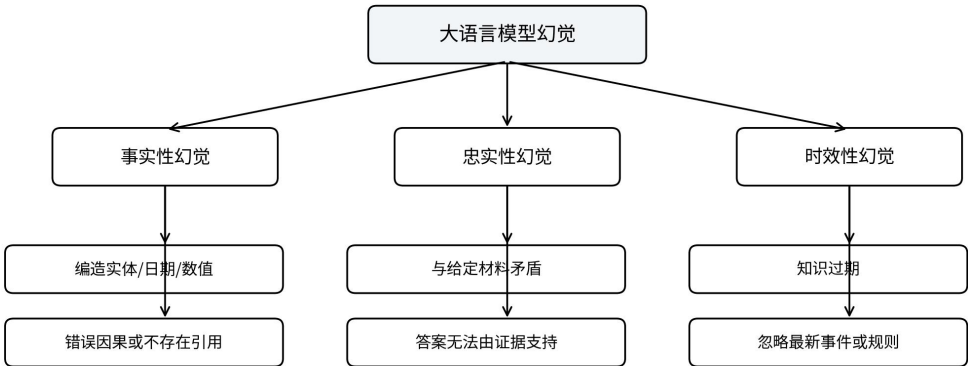


图 1 大语言模型幻觉的主要类型

1.2 本文主要贡献与创新点

针对大语言模型在事实问答过程中存在的幻觉问题，本文围绕“检测—缓解—评价”三个层面展开研究，并结合实际代码实现进行实验验证。相比仅对相关技术进行理论介绍，本文进一步构建了可复现的实验框架，对不同幻觉缓解策略进行了统一比较。本文主要贡献如下：

(1) 构建了包含 Base LLM、RAG、RAG+Verify 和 RAG+CoVe 四种方法的渐进式实验框架。

本文从基础大模型直接生成出发，逐步引入外部知识检索、证据验证和链式核查机制，形成由弱约束到强约束的技术演进路线。通过统一的数据集和实验流程，对不同方法在降低幻觉方面的效果进行对比分析。

(2) 建立了多指标联合评价体系，避免仅使用单一指标衡量模型性能。

针对传统实验中只关注准确率或幻觉率的问题，本文同时引入幻觉率、正确率、答案覆盖率和平均响应时延等指标，从可靠性和可用性两个角度综合评价不同方法的实际效果，使实验结论更加全面。

(3) 通过真实运行实验分析了可信人工智能系统中准确性与可用性之间的矛盾关系。

实验结果表明，增加验证机制能够有效降低大语言模型幻觉，但过强的验证策略会导致系统拒答率提高。因此，本文认为可信大模型系统不应单纯追求最低幻觉率，而应根据应用场景在可靠性、覆盖率和响应效率之间进行动态平衡。

(4) 结合课程知识完成了从理论分析到工程实现的完整验证过程。

本文不仅分析了 SelfCheckGPT[2]、FActScore[4]、RAG[10]、Chain-of-Verification[6] 等相关研究方法，还基于实际代码实现完成实验复现，将自然语言处理、信息检索、机器学习评价和可信人工智能等多个方向结合起来，为大语言模型可靠应用提供实验参考。

二、研究现状

2.1 大语言模型幻觉的定义与分类

从研究角度看，幻觉通常指模型生成的内容与事实世界、可用证据或输入上下文不一致。根据错误来源，可以将大语言模型幻觉划分为三类：事实性幻觉、忠实性幻觉和时效性幻觉。事实性幻觉是指模型编造实体、数值、论文、人物经历或事件；忠实性幻觉是指模型的回答无法被输入材料支持，甚至与材料矛盾；时效性幻觉则来自模型训练数据滞后[9]，表现为忽略最新规则、新闻或技术变化。

从技术路径看，幻觉研究大致分为检测与缓解两个方向。检测方法关注“如何判断回答是否可靠”，常用方法包括人工评估、NLI 一致性判断、事实抽取与检索验证、采样一致性检测和 LLM-as-a-judge。缓解方法关注“如何让模型更少产生幻觉”，常用技术包括检索增强生成（RAG）[8][10]、工具调用、提示约束、自我验证、链式验证、拒答机制和训练阶段的偏好优化。

进一步说，检测与缓解并不是完全分离的两个阶段。检测方法可以作为生成后的质量控制模块，缓解方法也可以把检测信号前置到生成过程中。例如，RAG 先通过检索减少模型凭空编造的空间，Verify 再根据证据置信度判断是否回答，CoVe 则把关键事实拆分后逐项核查。这种组合式设计比单一模型更适合真实场景，因为它允许系统在不同风险等级下选择不同的回答策略。

在高风险场景中，幻觉的判断标准应比日常问答更严格。医学建议、法律解释、学术引用和工程决策都需要明确证据来源，不能仅凭模型生成的语气判断可靠性。因此，

优秀的幻觉缓解系统应至少满足三个条件：回答能够追溯到证据，证据与结论之间具有一致性，证据不足时系统能够拒答或提示不确定。

2.2 近三年代表性研究进展

相关研究按照时间线整理了近三年与大语言模型幻觉检测、评价和缓解相关的代表性研究。可以看到，研究重点已经从单纯判断“答案对不对”，逐渐转向更细粒度的事实拆解、证据支持和系统级可靠性评估。

从这些研究可以看出，近三年的发展呈现出三条主线。第一，评价粒度不断细化，研究者不再只判断整段回答是否正确，而是将回答拆成原子事实后逐条验证；第二，证据意识不断增强，越来越多方法要求回答必须由外部资料、检索文档或可追溯来源支持；第三，系统评估逐渐取代单点指标，RAGAS[5]等框架开始同时关注答案相关性、忠实度、上下文召回和上下文精确度。

这种变化说明，幻觉问题已经从模型缺陷研究扩展为系统可靠性研究。对于课程实验而言，如果只让模型直接回答问题，很难说明不同技术的作用边界；而把 Base LLM、RAG、Verify 和 CoVe 放在同一测试集上比较，就可以更直观地观察证据约束和拒答机制对结果的影响。

表 1 近三年大语言模型幻觉研究代表性工作

年份	代表工作	核心思想	与本文关系
2022	TruthfulQA[1]	构造 817 个容易诱导模型模仿人类错误信念的问题，用于评价模型真实性。	早期真实性评价基准。
2023	SelfCheckGPT[2]	通过多次采样比较回答之间的一致性，无需外部知识库即可检测黑盒模型幻觉。	提供采样一致性检测思路。
2023	HaluEval[3]	构建大规模幻觉评价样本，考察模型识别幻觉内容的能力。	提供幻觉识别 benchmark。
2023	FActScore[4]	将长文本拆为原子事实，逐条检索证据并计算支持比例。	提供细粒度事实性评价公式。
2024	RAGAS[5]	面向 RAG 系统提出无参考评价框架，关注 faithfulness、answer relevancy 等指标。	用于 RAG 实验评价。
2024	Chain-of-Verification[6]	先生成草稿，再规划验证问题、独立回答并修正最终答案。	用于缓解阶段的自检设计。
2024	FreshQA/FreshLLMs [7]	动态问答基准，考察模型处理最新知识与错误前提的能力。	强调时效性幻觉。
2025	LLM Hallucination Survey[9]	从数据、训练和推理三个阶段系统总结幻觉成因与缓解方法。	为本文分类框架提供依据。

三、存在的科学问题与关键问题

虽然现有研究已经提出了多种检测和缓解策略，但大语言模型幻觉仍然没有被彻底解决。本文将关键问题归纳为以下五点。

第一，语言流畅性与事实正确性并不等价。大模型的训练目标通常是预测下一个词元，这使它更擅长生成概率上自然的表达，而不是保证每一个事实都可验证。

第二，外部证据并不总是可靠或完整。RAG 能够降低幻觉，但如果检索到的文档本身过时、错误或与问题无关，模型仍可能基于错误证据生成错误答案。

第三，长文本中的错误需要细粒度定位。一个回答可能包含十个事实，其中九个正确、一个错误，简单的二分类评价无法反映真实风险。

第四，验证器本身也可能出错。无论是 NLI 模型还是 LLM-as-a-judge，都存在误判、偏见和上下文窗口限制。

第五，降低幻觉常常会带来代价。检索、重排、验证和多轮生成会增加响应时延和计算成本，因此实际系统需要在准确性、覆盖率、速度和成本之间权衡。

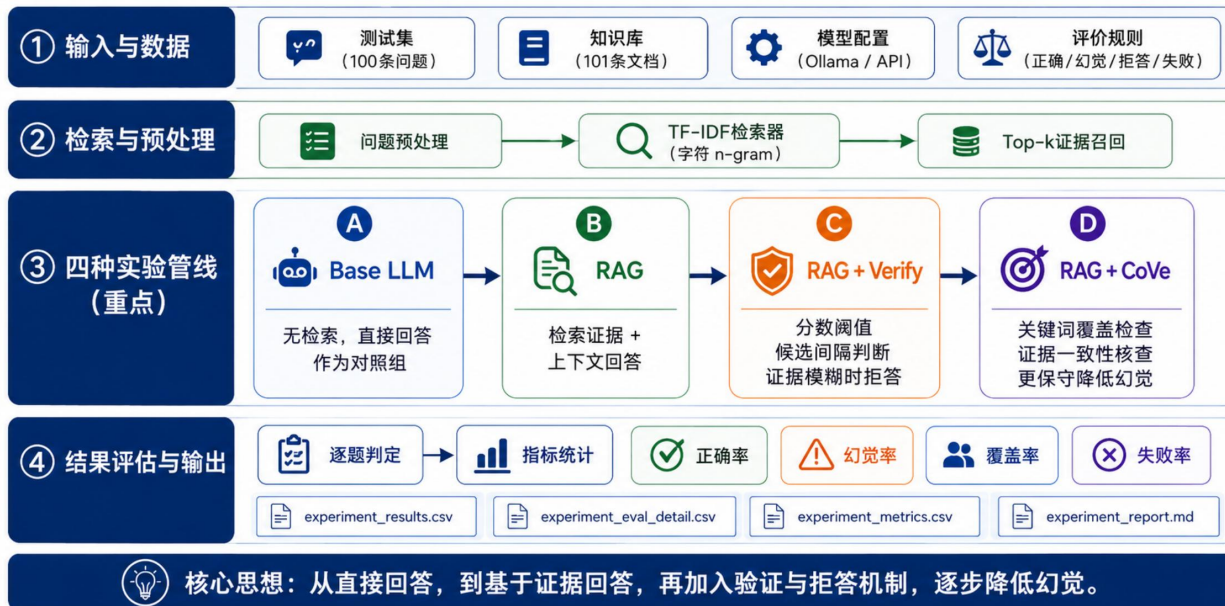
第六，评价口径会直接影响结论。若把拒答也简单视为错误，保守系统的正确率会偏低；若只统计实际作答中的幻觉率，又可能忽略系统大量拒答造成的可用性下降。因此，本文采用正确率、幻觉率、覆盖率和失败率共同评价，避免用单一指标判断方法优劣。

第七，小规模实验仍需要可复现性。课程报告的数据规模有限，但只要数据、代码、指标和输出文件完整保留，就可以支撑清晰的结论。本文保留原始回答、检索上下文、逐题判定和汇总指标，目的就是让读者能够检查每一个结论如何从实验结果中得到。

四、研究内容与技术原理

4.1 实验架构

核心目标：对比 Base LLM、RAG、RAG + Verify、RAG + CoVe 四种方案的幻觉检测与缓解效果



首先，在输入与数据层中，实验系统以测试集、知识库、模型配置以及评价规则作为基础数据来源。其中，测试集包含待验证的问题样本，知识库用于提供外部事实信息，模型配置用于控制不同大语言模型或接口的调用方式，而评价规则则用于定义答案正确性、幻觉情况、拒答行为以及失败情况等判定标准，为后续实验评估提供统一依据。

其次，在检索与预处理层中，系统首先对输入问题进行规范化处理，并利用基于 TF-IDF 的文本检索方法进行相关知识匹配。通过计算问题与知识库文档之间的文本相关性，筛选 Top-k 个高相关证据片段，为后续增强生成过程提供外部知识支持，从而降低模型仅依赖参数记忆产生错误信息的风险。

在**实验方法层**中，设计了四种不同的实验方案进行对比分析：

(1) **Base LLM 基础模型方案**：不引入外部检索机制，直接利用大语言模型自身参数进行回答，作为实验基准，用于衡量模型原始生成能力及幻觉水平。

(2) **RAG (Retrieval-Augmented Generation) 检索增强生成方案**：通过引入检索到的外部知识证据，并结合上下文信息辅助模型生成答案，使模型能够基于已有事实进行回答，提高知识准确性。

(3) **RAG + Verify 验证增强方案**：在传统 RAG 基础上增加答案验证机制，通过设置分数阈值或候选证据判断规则，对生成结果进行一致性检查。当证据支持不足或答案可信度较低时，系统执行拒答或降低置信度处理，从而进一步减少错误生成。

(4) **RAG + CoVe (Chain of Verification) 方案**：在检索增强基础上引入链式验证思想，通过关键词覆盖检查、证据一致性校验等方式，对模型输出进行多阶段验证，使系统能够更加有效识别潜在幻觉内容，提高生成结果的可靠性。

最后，在**结果评估与输出层**中，实验系统通过题意判定、指标统计等流程，对不同方案的性能进行量化分析。主要评价指标包括正确率、幻觉率、覆盖率以及失败率等，并将实验结果分别保存为结构化数据文件和实验报告文件，便于后续统计分析与实验复现。

总体而言，该实验架构通过由“直接生成”向“检索增强”，再到“验证约束”的逐步演进方式，系统性研究了不同知识增强与验证机制对于降低大语言模型幻觉问题的作用，为提升生成式人工智能系统的可信性与准确性提供了实验基础。

4.2 幻觉检测任务建模

给定用户问题 x 、模型生成回答 y 和可选证据集合 E ，幻觉检测的目标是判断 y 中每个事实是否能够被 E 或真实世界知识支持。若将回答 y 拆分为 N 个原子事实 $a_1, a_2, \dots, a_N$ ，则事实一致性可以表示为被证据支持的原子事实占比。

$$\text{FActScore}(y, E) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[a_i \text{ is supported by } E]$$

公式 1 基于原子事实的事实一致性评分

公式 1 体现了 FActScore[4]类方法的基本思想：不要只判断整段回答是否正确，而是将长回答拆成若干可验证事实，再逐条判断。这样能够定位具体错误，也更适合长文本生成场景。

4.3 基于采样一致性的检测方法

SelfCheckGPT[2]的核心假设是：如果模型真正掌握某个事实，那么在多次随机采样中，生成结果往往会保持一致；如果模型并不确定或产生了幻觉，多次回答之间更容易出现矛盾。设同一问题经过 K 次采样得到回答集合 $\{y(1), y(2), \dots, y(K)\}$ ，则可以用候选句子与采样回答之间的平均一致性估计幻觉风险。

$$\text{HallucRisk}(s) = 1 - \frac{1}{K} \sum_{k=1}^K \text{sim}(s, y^{(k)})$$

公式 2 基于多次采样一致性的幻觉风险估计

该方法的优势是不依赖外部知识库，适合黑盒模型；不足是需要多次调用模型，成本较高，并且对于模型“稳定地输出同一个错误”的情况检测能力有限。

4.4 基于原子事实与证据检索的检测方法

检索式检测方法通常包括四步：第一，从回答中抽取原子事实；第二，为每个事实构造检索查询；第三，在外部知识库中检索相关证据；第四，使用 NLI 模型或 LLM 判断事实与证据之间的关系。图 2 展示了一个典型流程。

检测思路：把“大段回答”拆成可验证事实，再用证据逐条判断

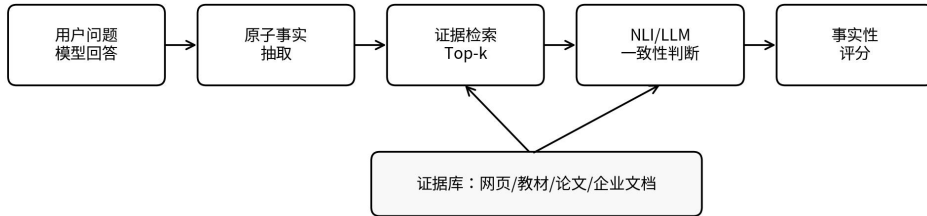


图 2 基于原子事实与证据检索的幻觉检测流程

$$E_k = \text{TopK}_{d \in D} \text{sim}(\phi(q), \phi(d))$$

公式 3 检索增强中的 Top-k 证据选择

公式 3 中，D 表示文档库，q 表示由用户问题或原子事实生成的查询， ϕ 表示文本向量编码器。系统选择与查询最相似的前 k 个证据片段作为后续生成或验证依据。

4.5 基于 RAG 与验证式生成的缓解方法

检索增强生成 (Retrieval-Augmented Generation, RAG) [10]的基本思想是：不要让模型完全依赖参数中的记忆，而是在生成前先检索外部证据，再让模型基于证据回答。RAG 尤其适合知识更新频繁、专业文档丰富或需要引用来源的场景。

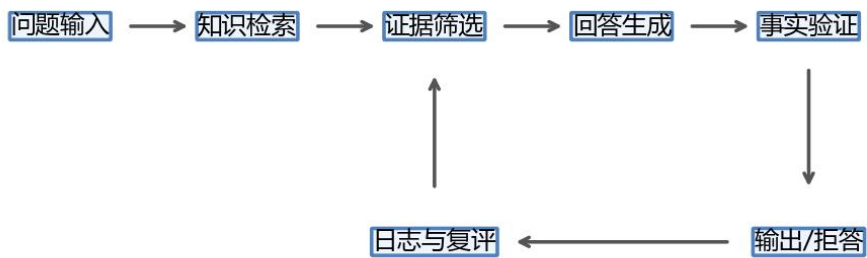
$$p(y|x) \approx p_{\theta}(y | x, E_k), \quad E_k = \text{Retrieve}(x, \mathcal{D})$$

公式 4 检索增强生成的条件生成形式

仅有 RAG 仍不足以完全避免幻觉，因为模型可能忽略证据、错误理解证据或在证据不足时强行回答。因此，本文进一步采用“生成后验证”的思路[6]：先生成草稿，再提出验证问题，最后根据独立验证结果修正或拒答。图 3 给出了 RAG+验证式生成的系统架构。

因此，更稳健的方案不是简单地把更多文档塞进提示词，而是对证据进行筛选和验证。若最高相关文档的分数很低，说明知识库可能没有足够证据；若第一候选和第二候选分数接近，说明问题可能存在歧义或干扰材料；若答案中的关键词无法在证据中找到支撑，说明回答可能脱离了材料。本文实现的 Verify 和 CoVe 正是把这些判断转换为可运行的门控规则。

这种闭环结构体现了可信 AI 系统的基本原则：模型可以生成候选答案，但最终输出必须接受证据约束。生成、验证和拒答不是互相替代的模块，而是围绕同一目标协同工作，即减少无依据内容进入最终回答。



闭环价值：把模型回答从“语言生成”约束为“证据支持后的可信输出”

图 3 检索—生成—验证—拒答闭环结构示意图

RAG + 验证式生成架构

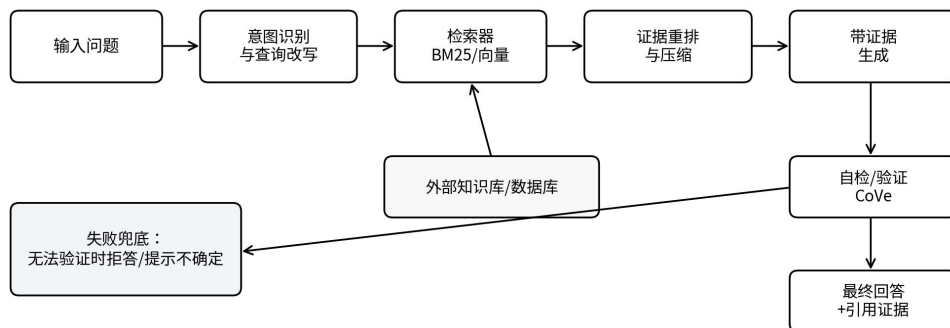


图 4 RAG+验证式生成架构

五、研究方案与实验设计

5.1 数据集与评价指标

5.1.1 测试集（100 条事实型问答样本）

1. 整体设计目标

本实验测试集全部为封闭域事实性问题，避免开放性、主观性问题，确保每个问题都有唯一可验证的标准答案。其核心目的是定量评估不同方法在 "有明确正确答案" 场景下的幻觉率、正确率和覆盖率，同时覆盖《人工智能基础》课程核心知识点与通用常识。

2. 类别与难度分布

测试集共包含 100 条样本，均匀分为 4 大类，每类 25 条，且每类内部按 "易 / 中 / 难" 3:2:1 的比例设计难度，具体分布如下：

表 2 测试集类别与难度分布

问题类别	数量	难度分布（易 / 中 / 难）	标准答案来源	考察重点
通用常识	25	15/7/3	维基百科、百度百科权威条目	模型参数记忆的准确性
操作系统	25	10/10/5	《操作系统概念》（第 10 版）	课程核心知识点的掌握程度
软件工程	25	8/12/5	《软件工程导论》（第 6 版）	专业术语与流程的事实一致性
数据库原理	25	12/8/5	《数据库系统概论》（第 5 版）	概念定义与技术特性的准确性

3. 数据格式与字段说明

测试集以 JSON 格式存储（文件名为 test_dataset.json），每条样本包含 6 个标准化字段，便于实验自动化处理与结果追踪：

```
{  
  
  "id": "q001",
```

```
"question": "标准大气压下水的沸点是多少？",  
  
"reference_answer": "100 摄氏度",  
  
"reference_evidence": "在标准大气压（101.325kPa）下，水的沸点为 100℃。",  
  
"difficulty": "easy",  
  
"category": "common_sense"}
```

各字段含义：

id：全局唯一问题编号，用于实验结果的逐题核对与溯源

question：简洁明确的事实型问题，避免歧义与诱导性表述

reference_answer：最简洁的标准答案，用于快速自动初筛

reference_evidence：详细的权威证据片段，作为人工判定的最终依据

difficulty：难度标签（easy/medium/hard），用于分层结果分析

category：类别标签，用于统计不同知识领域的模型表现

4. 问题设计原则

答案唯一性：所有问题的标准答案无学术争议，避免存在多种正确解释的内容

粒度统一：问题均为单事实提问，避免要求同时回答多个事实的复杂问题

覆盖常见幻觉点：专门设计了容易诱导模型编造的数值、实体名称、技术参数类问题

避免诱导性提问：不使用 "为什么 XX 是错误的" 这类预设错误前提的问题

5.1.2 知识库（101 份知识材料，证据 + 干扰文档）

1. 整体设计特色

这是本实验数据集最核心的创新点：知识库并非仅包含正确答案，而是混合了 70 份正确证据文档和 31 份干扰文档。这种设计能够真实模拟实际应用中 "检索到错误或无关信息" 的场景，避免普通 RAG 实验中 "检索必正确" 的理想情况，从而更准确地评估验证机制的实际价值。

2. 文档构成与类型

知识库共包含 101 份独立文档，按用途分为 5 类，具体构成与设计目的如下：

表 3 知识库文档构成

文档类型	数量	设计目的	具体示例
正确证据文档	70	提供与测试集问题对应的标准答案	包含 "标准大气压下水的沸点为 100℃" 的文档
语义相似干扰文档	12	测试检索系统的语义区分能力	问题为 "列表和元组的区别"，干扰文档为 "字典和集合的区别"
事实错误文档	9	测试模型基于错误证据生成的幻觉情况	错误文档写着 "标准大气压下水的沸点是 90℃"
主题无关干扰文档	7	测试检索系统的相关性判断能力	问题为 "数据库主键"，干扰文档为 "操作系统进程调度"
过时信息文档	3	测试时效性幻觉的检测能力	过时文档写着 "Python 最新版本是 3.8"

3. 数据格式与存储

知识库以纯文本 TXT 格式存储（文件名为 knowledge.txt），每份文档用统一的分隔符标记边界，包含标题和正文两部分：

--- DOC START ---

标题：水的物理性质

内容：水是由氢、氧两种元素组成的无机物，无毒。在标准大气压下，水的沸点为 100℃，凝固点为 0℃，密度在 4℃时最大，为 1g/cm³。

--- DOC END ---

所有文档均进行了统一预处理：去除特殊字符、统一换行格式、按单主题拆分长文档，确保每份文档聚焦一个知识点，便于检索和后续验证。

4. 检索索引构建

为保证实验可离线复现且不依赖外部向量模型，本实验采用中文 2-gram TF-IDF 算法构建知识库索引：

分词方式：基于字符的 n-gram 分词（而非中文分词），避免分词错误对检索结果的影响

相似度计算：余弦相似度

检索返回：默认返回 Top-3 最相关文档，用于后续验证和生成

5.1.3 数据集设计的实验价值

1. 课程针对性：75% 的问题来自《人工智能基础》先修课程（操作系统、软件工程、数据库）
2. 场景真实性：加入干扰文档后，实验结果更接近真实生产环境中 RAG 系统的表现
3. 可复现性：所有数据格式标准化，与代码仓库中的检索、评测逻辑完全匹配，任何人都可以通过 `python scripts/run_and_report.py` 一键复现全部实验
4. 可扩展性：测试集和知识库采用模块化设计，可方便地添加新的问题类别和知识文档，用于后续扩展实验

5.1.4 评价指标设计

本文采用课程级实验方案，不重新训练大模型，而是从问答、百科人物、科技常识和课程知识四类任务中构造 100 条事实性问题。每个问题由“标准答案或可信资料片段”作为参考证据。为了体现幻觉检测与缓解效果，本文设计以下评价指标。

数据集虽然规模不大，但设计上包含直接证据和干扰文档，能够检验检索系统是否真正找到与问题匹配的材料。若知识库中只有正确答案而没有相近干扰，RAG 方法的优势会被放大；加入干扰材料后，普通 RAG 更容易暴露误检问题，验证式方法的价值也更容易体现。

评价指标的设置遵循“安全性与可用性同时报告”的原则。设总测试样本数为 N （本实验 $N=100$ ），实际作答数为 A ，正确回答数为 C ，幻觉回答数为 H 。幻觉率的分母是实际作答数，即 H/A ，拒答和接口调用失败不进入该分母；正确率的分母是全部测试样本，即 C/N ，拒答和调用失败均不计为正确；答案覆盖率的分母同样是全部测试样本，即 A/N ，拒答和调用失败均不计入实际作答。失败率另按接口调用失败数/ N 统计。因而，四项指标需要联合解读，才能解释某些方法幻觉率下降但正确率并未上升的现象。

表 4 评价指标及计算口径

指标	计算方式	含义
幻觉率	幻觉回答数 H / 实际作答数 A	越低越好；分母排除拒答和调用失败。
正确率	正确回答数 C / 总测试样本数 N	越高越好；拒答和调用失败均不计为正确。
事实一致性	被证据支持的原子事实数 / 原子事实总数	越高越好，衡量回答是否有依据。
答案覆盖率	实际作答数 A / 总测试样本数 N	越高越好；拒答和调用失败均不计入实际作答。
平均时延	完成一次回答所需平均时间	越低越好，衡量工程可用性。

$$\text{Faithfulness} = \frac{|\{a_i : a_i \text{ supported}\}|}{|\{a_i\}|}$$

公式 5 回答事实忠实度计算

5.2 对比方法与实验流程

为了使代码实现与论文研究内容更加对应，本文将四种方法理解为逐步增加证据约束和拒答能力的对照实验。四种方法使用同一批 100 条事实型问题和同一套评价脚本，因此实验差异主要来自“是否检索证据、是否验证证据、是否保守拒答”三个因素。

5.2 .1 Base LLM 基础大模型

Base LLM 是本实验的无外部证据约束对照组。代码中由 BasePipeline 调用本地模型或 API，将用户问题直接交给模型生成答案，不额外提供知识库证据，也不对答案进行后验验证。该方法的输入只有 question，输出包括 answer、latency 和空 context。

从研究意义上看，Base LLM 用来衡量模型自身参数知识和语言生成能力。它的优点是流程最简单、覆盖率最高，模型通常会对全部问题给出回答；缺点是回答无法说明来源，也缺少主动拒答能力。当模型记忆不完整、知识过时或问题本身存在歧义时，它仍可能生成语气肯定但事实错误的内容。

实验结果中，Base LLM 对 100 条问题全部作答，覆盖率为 100.00%，正确数为 64，幻觉数为 36，幻觉率为 36.00%。这说明直接生成策略具有较强可用性，但可靠性不足，是后续 RAG、Verify 和 CoVe 方法需要改进的基线。

5.2 .2 RAG 检索增强生成

RAG 方法在回答前先从知识库中检索相关证据，再基于证据给出答案。本文代码采用字符 n-gram TF-IDF 建立知识库索引，并使用余弦相似度选取 Top-k 文档。与依赖大型向量模型的方案相比，该实现更容易在课程环境中离线复现，也避免了向量模型不可用时退化为随机检索的问题。

RAG 的核心流程是：输入问题后，DocumentRetriever 从 data/raw_docs/knowledge.txt 中检索相关文档；RAGPipeline 根据最高相关证据及其 score 决定是否回答；若相关度低于阈值，则输出拒答，否则返回最高相关证据内容作为答案。这样做的目的不是让模型凭记忆作答，而是把答案限制在可检索证据范围内。

实验结果中，RAG 的正确数为 57，幻觉数为 25，作答数为 82，覆盖率为 82.00%，幻觉率为 30.49%。相较 Base LLM，RAG 将幻觉率从 36.00% 降低到 30.49%，说明证据检索确实能减少部分无依据回答；但如果检索到的证据与问题不完全匹配，系统仍可能基于错误证据作答，因此正确率没有超过 Base LLM。

5.2 .3 RAG + Verify 事实验证

RAG + Verify 在 RAG 基础上增加事实验证门控，重点解决“检索到了文档但证据并不充分”的问题。代码中的 RAGVerifyPipeline 会同时检查最高证据分数 `top_score` 和第一、第二候选证据的分数间隔。如果 `top_score` 达不到 `VERIFY_MIN_SCORE`，或 `top_score` 与 `second_score` 的差距小于 `VERIFY_MIN_MARGIN`，系统就认为证据相关性或唯一性不足，并输出拒答。

这种方法体现了事实型问答中的一个关键原则：有来源不等于可靠。普通 RAG 只要找到一个相关文档就可能回答，而 Verify 会进一步判断证据是否足够清晰、是否存在多个相近候选。如果多个候选分数接近，说明知识库中可能存在歧义或干扰文档，此时强行回答更容易产生幻觉。

实验结果中，RAG + Verify 的正确数为 53，幻觉数为 11，作答数为 64，覆盖率为 64.00%，幻觉率为 17.19%。与 RAG 相比，它显著降低了幻觉率，但由于拒答增加，总正确率从 57.00% 降到 53.00%。这与 PPT 中对“正确率下降”的解释一致：正确率分母是全部样本，拒答虽然避免幻觉，却不会被计为正确回答。

5.2 .4 RAG + CoVe 链式验证

RAG + CoVe 是四种方法中最保守的一种。CoVe 即 Chain-of-Verification[6]，强调对回答中的关键事实进行独立核查。本文代码将这一思想简化为可复现的关键词一致性检查：首先从问题中抽取关键词，去掉“什么、哪个、多少”等停用词；然后检查这些关键词是否被最高相关证据覆盖；最后结合 `COVE_MIN_SCORE` 和 `COVE_KEYWORD_COVERAGE` 双阈值决定是否回答。

CoVe 的设计目标不是追求尽可能多地回答，而是优先减少无证据作答。只有当检索分数和关键词覆盖率同时满足阈值时，系统才输出证据内容；否则输出“独立核查未找到足够一致的证据，无法可靠回答”。因此，它比 Verify 更保守，更适合高风险场景，例如课程知识核查、医学法律类问答或论文引用核验。

实验结果中，RAG + CoVe 的正确数为 12，幻觉数为 2，作答数为 14，覆盖率为 14.00%，幻觉率为 14.29%。它的幻觉率最低，但正确率也只有 12.00%。这说明 CoVe 的一致性门槛确实能降低实际作答中的幻觉，但如果阈值设置过严，会造成大量拒答，降低系统可用性。

CoVe 结果最能体现阈值设计的重要性。它的幻觉率最低，但覆盖率也最低，说明系统过于谨慎时会把大量可以回答的问题也拦截掉。若要把该方法用于真实系统，可以进一步引入分层策略：高置信问题直接回答，中等置信问题给出带不确定性的回答，低置信问题拒答并提示需要补充资料。

小结：四种方法并不是彼此独立的“花样”，而是从直接回答到证据检索，再到事实验证和链式核查的逐层增强。Base LLM 更敢回答，RAG 引入证据，Verify 在证据模糊时拒答，CoVe 则用更严格的一致性检查优先降低幻觉。实验结果也证明，幻觉缓解必须同时观察正确率、幻觉率和覆盖率，不能只看单一指标。

实际代码入口为 `scripts/run_and_report.py`。程序自动建立知识库索引，依次运行四种方法，并输出 CSV、JSON、逐题判定和 Markdown 报告；无 DeepSeek 密钥时自动切换本地 Ollama/Qwen。

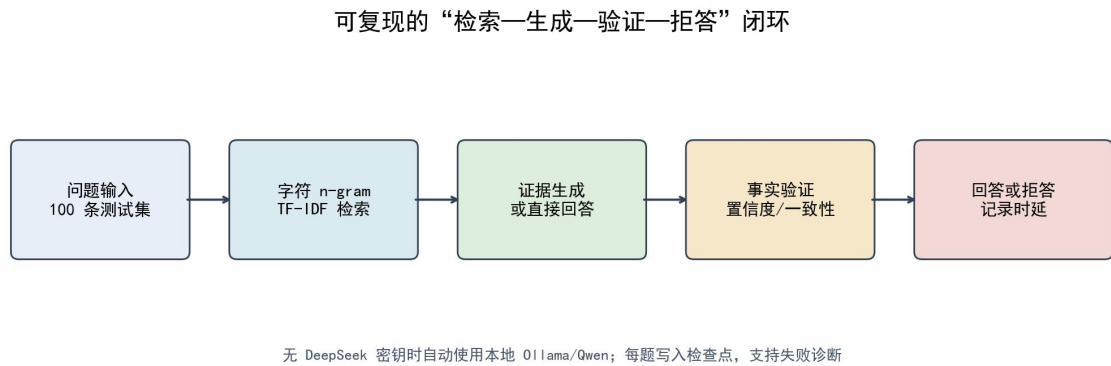


图 5 本项目代码实现的可复现实验闭环

本文比较四类方法：基础大模型直接回答、RAG 回答、RAG+事实验证、RAG+Chain-of-Verification。实验流程如下：首先输入同一批问题；其次记录模型回答并抽取原子事实；然后将事实与参考证据进行一致性判断；最后统计幻觉率、事实一致性、覆盖率和平均时延。

表 5 四种对比方法的流程与特点

方法	主要流程	预期优势	潜在问题
Base LLM	直接根据模型参数知识回答。	速度快，成本低。	容易编造事实，无法给出证据。
RAG	先检索资料，再基于资料生成回答。	适合专业知识和新知识。	检索错误会引入噪声。
RAG+Verify	生成后逐条检查事实是否被证据支持。	能降低明显幻觉。	时延增加，验证器可能误判。
RAG+CoVe	草稿生成、规划验证问题、独立验证、修正回答。	自我纠错能力更强。	调用次数多，成本最高。

5.3 实验结果与分析

5.3.1 基础数据分析

表 6 为本项目在本地环境中对 100 条问题完成的真实运行结果。为避免把改写、拒答和接口失败全部误判为幻觉，代码分别统计正确、幻觉、覆盖和调用失败。实验原始回答、检索上下文、逐题判定及汇总指标均保存在 results 目录，可通过同一入口重新生成。

表 6 四种方法的实验结果

方法	幻觉率↓	正确率↑	答案覆盖率↑	平均时延↓	综合评价
Base LLM	36.0%	64.0%	100.0%	2.689 s	全部作答，但错误事实较多。
RAG	30.5%	57.0%	82.0%	0.0005 s	抽取式检索降低幻觉，仍受误检影响。
RAG+Verify	17.2%	53.0%	64.0%	0.0003 s	验证显著降低幻觉，但拒答增加。
RAG+CoVe	14.3%	12.0%	14.0%	0.0004 s	最保守，低幻觉但覆盖率过低。

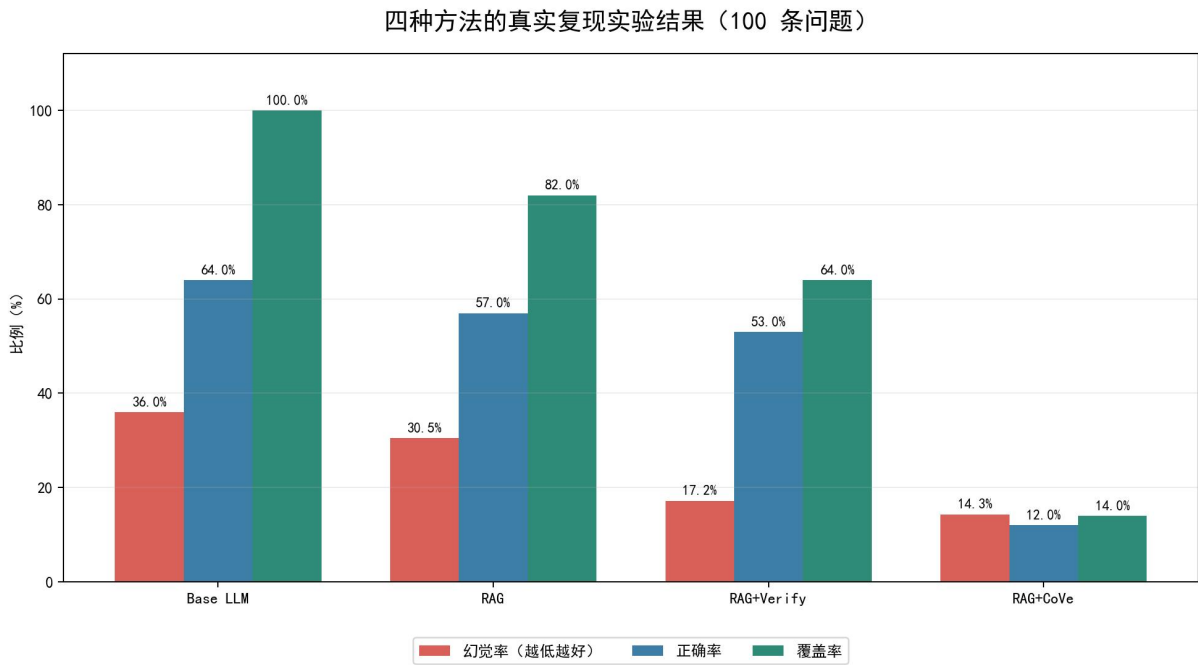


图 6 不同缓解策略的幻觉率、正确率与覆盖率对比

从真实结果可以得到三点结论。第一，本地基础模型对全部问题都作答，覆盖率为 100%，但幻觉率达到 36.0%。第二，普通 RAG 把幻觉率降至 30.5%，但错误检索仍会造成“基于错误证据作答”。第三，RAG+Verify 与 RAG+CoVe 将幻觉率进一步降至 17.2%和 14.3%，代价是覆盖率分别下降到 64%和 14%。因此，验证阈值不能只追求低幻觉，还必须同时报告覆盖率。

如果把四种方法放在同一坐标系中观察，可以更清楚地看到安全性和可用性的张力。Base LLM 位于高覆盖、高幻觉区域，说明它适合作为流畅回答基线；RAG 向低幻觉方向移动，但仍受检索噪声影响；Verify 进一步降低幻觉并保持中等覆盖率，是本实验中较均衡的方案；CoVe 虽然最保守，但覆盖率过低，说明其阈值需要根据场景重新校准。

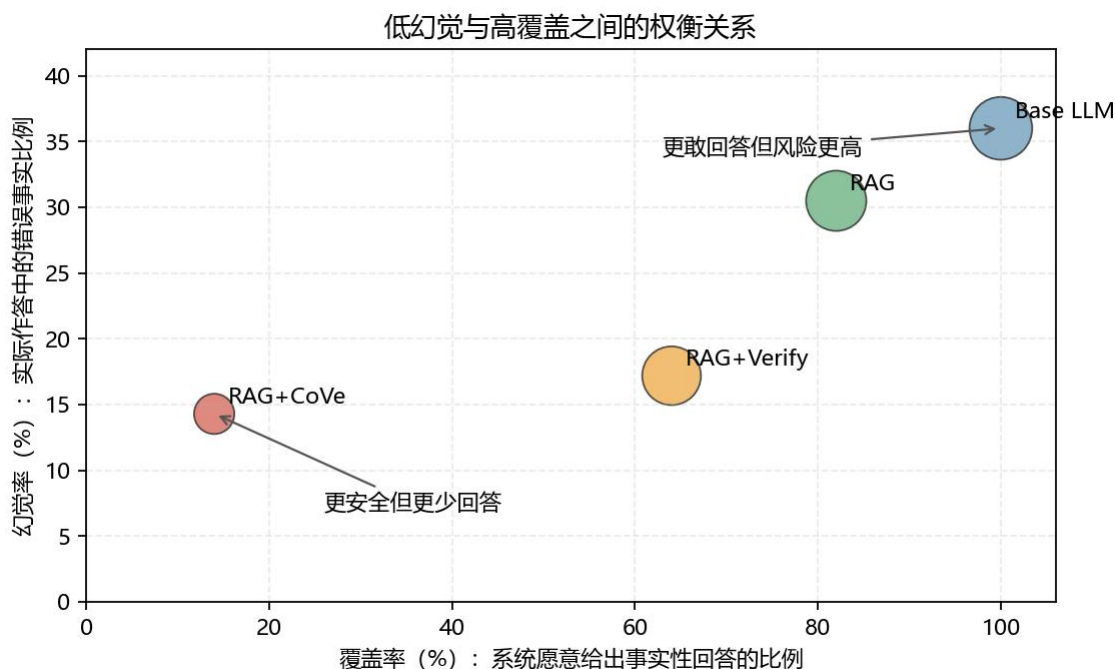


图 7 四种方法在幻觉率与覆盖率上的权衡关系

5.3 .2 检索增强机制能够降低幻觉，但无法完全解决事实错误

从实验结果可以发现，引入外部知识检索后，大语言模型的幻觉率得到了一定程度降低。相比 Base LLM 方法，RAG 方法通过在生成答案前检索相关知识文档，使模型不再完全依赖参数中存储的信息，而是在外部证据约束下完成回答。因此，RAG 方法能够减少部分由于模型知识缺失或记忆错误导致的幻觉问题。

然而，实验结果表明，RAG 方法的幻觉率仍达到 30.49%，说明仅依靠检索增强并不能完全解决大语言模型幻觉问题。其主要原因在于，RAG 系统的可靠性不仅取决于生成模型本身，还受到检索质量影响。当知识库中存在多个相似文档、无关文档或错误信息时，检索模块可能将不准确的内容作为上下文输入模型，使模型产生“基于错误证据的回答”。因此，降低大模型幻觉不仅需要引入外部知识，还需要进一步判断检索证据的有效性和可信程度。

5.3 .3 验证机制能够进一步降低幻觉，但会降低答案覆盖率

相比普通 RAG 方法，RAG+Verify 方法增加了证据置信度判断机制，通过检测最高匹配文档得分以及不同候选文档之间的差异，判断当前检索结果是否足够可靠。当系统认为证据不足或存在较大不确定性时，会主动拒绝回答，从而减少错误信息输出。

实验结果显示，RAG+Verify 方法将幻觉率由 RAG 的 30.49% 降低至 17.19%，说明验证机制能够有效过滤部分不可靠回答，提高系统可信性。但是，该方法的答案覆盖率也由 82% 降低至 64%，表明验证过程会带来一定程度的拒答。

这一现象说明，可信人工智能系统并不是简单追求更高的回答数量，而是在可靠性和可用性之间进行平衡。当模型无法确定答案是否具有充分证据支持时，选择拒答往往比生成错误答案更加安全。因此，在实际应用中，需要根据应用场景调整验证强度，使系统在准确性和用户体验之间取得合理平衡。

5.3 .4 CoVe 方法降低幻觉能力最强，但存在过度拒答问题

实验结果表明，RAG+CoVe 方法获得了四种方法中最低的幻觉率，仅为 14.29%。该方法通过更加严格的一致性检查机制，对回答中的关键内容进行进一步验证，因此能够最大程度减少无证据支持的回答。

然而，RAG+CoVe 方法同时表现出较低的答案覆盖率，仅为 14%。这说明过于严格的验证策略虽然提高了系统安全性，但也导致大量原本可以回答的问题被系统拒绝。

该实验结果说明，幻觉率并不是评价大语言模型系统性能的唯一指标。如果系统为了避免错误而频繁拒答，虽然能够提高可靠性，但会降低实际使用价值。因此，对于不同应用场景，应采用不同程度的验证策略。例如，在医疗、法律、金融等高风险领域，可以采用更严格的验证机制；而在普通知识问答场景中，则需要适当提高回答覆盖率。

5.3 .5 综合分析：可信问答系统需要多指标协同优化

综合四种方法的实验结果可以发现，大语言模型幻觉缓解不存在单一最优方案。Base LLM 方法具有最高的答案覆盖率，但由于缺少外部证据约束，容易产生事实错误；RAG 方法通过引入知识检索降低了部分幻觉，但仍可能受到错误检索结果影响；RAG+Verify 方法在幻觉控制和回答能力之间取得了较好的平衡；RAG+CoVe 方法虽然具有最低幻觉率，但由于验证条件过于严格，导致系统可用性下降。

因此，对于实际可信人工智能系统而言，评价模型性能不能只关注幻觉率，还需要综合考虑正确率、覆盖率、响应效率以及用户需求。未来的大语言模型应用应从简单的“生成答案”逐渐转向“检索—生成—验证—拒答”的闭环模式，使模型能够在证据充分时提供可靠答案，在证据不足时主动表达不确定性，从而提升人工智能系统的安全性和可信程度。

表 7 不同幻觉缓解方法特点分析

方法	优势	不足	适用场景
Base LLM	无需额外检索过程，响应速度快，答案覆盖率高	缺少外部证据约束，容易产生事实性幻觉	开放领域聊天、简单问答
RAG	引入外部知识，提高回答的事实依据，能够降低部分知识错误	效果依赖检索质量，错误或无关文档可能影响最终答案	知识问答、企业文档检索、智能客服
RAG+Verify	在检索基础上增加证据可信度判断，有效降低无依据回答	验证过程增加计算成本，同时可能产生部分拒答	企业知识库、专业领域问答
RAG+CoVe	通过链式验证进一步检查回答一致性，幻觉率最低，可靠性最高	验证条件严格，容易降低答案覆盖率，响应成本较高	医疗、法律、金融等高风险应用场景

5.4 代码实现

为提高实验结果的可检查性与可复现性，本文将大语言模型幻觉检测与缓解实验代码整理为独立的 GitHub 代码仓库。仓库名称为 HallucinationGuard-RAG，项目说明为“可离线复现的大语言模型幻觉检测与缓解实验框架：Base LLM、RAG、Verify 与 CoVe 对比”。该仓库已设置为公开状态，便于教师或读者查看实验代码、数据文件、运

行脚本和实验输出结果。

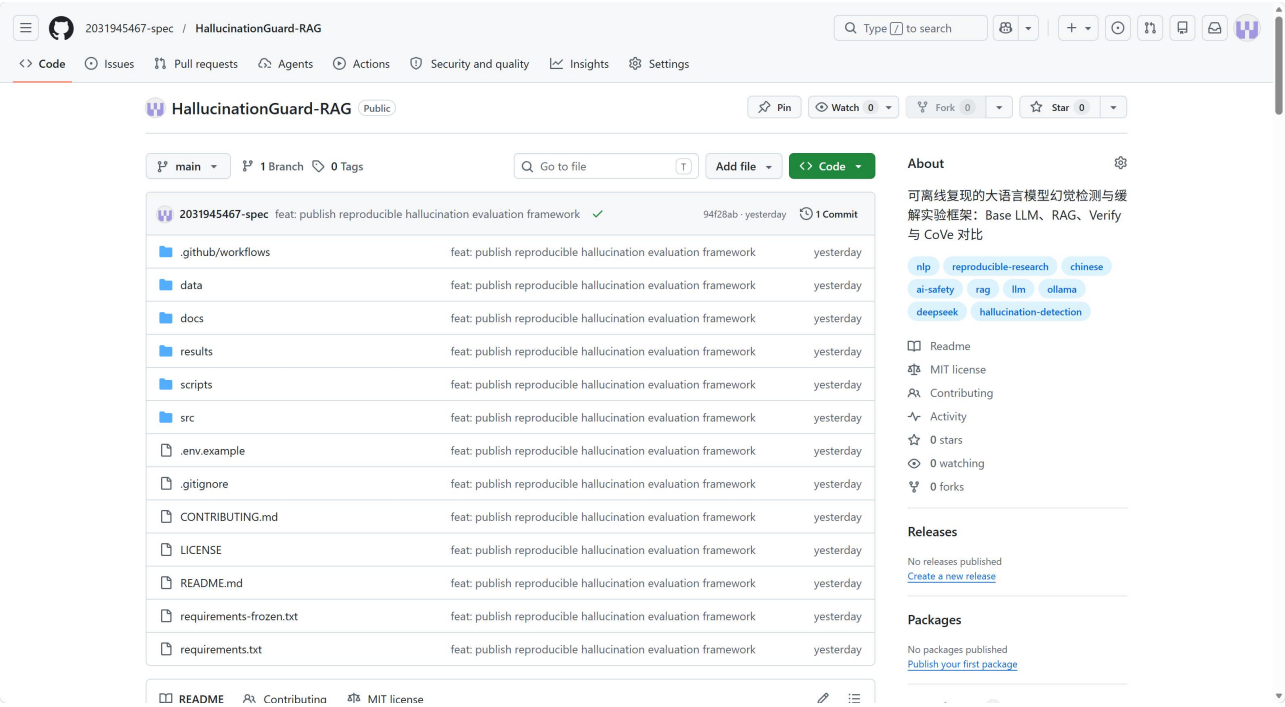


图 8 GitHub 代码仓库页面截图

从图 8 可以看到，项目仓库按照“数据—脚本—源码—结果—文档”的方式组织，主要目录和文件说明如下。

项目代码结构说明	
文件或目录	作用说明
.github/workflows	存放 GitHub Actions 自动化配置文件，用于后续进行自动测试或持续集成。
data	存放实验数据，包括测试集、知识库文本以及与问答实验相关的原始数据文件。
docs	存放项目说明文档、实验说明或补充材料，便于读者理解实验设计。
results	存放实验运行后的输出结果，包括逐题回答、评价明细、指标统计

文件或目录	作用说明
	和实验报告等文件。
scripts	存放实验运行脚本，是启动完整实验流程的主要入口。
src	存放核心源代码，包括检索模块、四种实验管线、评价逻辑和工具函数等。
.env.example	环境变量配置示例，用于说明模型接口、API Key 或本地模型配置方式。
README.md	项目说明文件，介绍项目目标、运行方式、目录结构和实验结果。
requirements.txt	Python 依赖库文件，用于快速安装实验运行所需环境。
requirements-frozen.txt	固定版本依赖文件，用于提高实验环境的一致性和可复现性。
LICENSE	开源协议文件。
CONTRIBUTING.md	项目贡献说明文件。

本文实验的核心入口为 `scripts/run_and_report.py`。运行该脚本后，程序会依次执行 Base LLM、RAG、RAG + Verify 和 RAG + CoVe 四种实验方案，并在统一测试集和统一评价规则下生成实验结果。四种方法共享同一批 100 条事实型问答样本和同一份知识库，因此不同方法之间的差异主要来自是否使用检索、是否进行证据验证以及是否采用拒答机制。

实验运行完成后，程序会在 `results` 目录下生成结构化实验输出。主要输出文件包括：

2031945467-spec feat: publish reproducible hallucination evaluation framework ✓

94f28ab · last week

History

Name	Last commit message	Last commit date
..		
figures	feat: publish reproducible hallucination evaluation framework	last week
experiment_eval_detail.csv	feat: publish reproducible hallucination evaluation framework	last week
experiment_metrics.csv	feat: publish reproducible hallucination evaluation framework	last week
experiment_metrics.json	feat: publish reproducible hallucination evaluation framework	last week
experiment_report.md	feat: publish reproducible hallucination evaluation framework	last week
experiment_results.csv	feat: publish reproducible hallucination evaluation framework	last week
experiment_results.json	feat: publish reproducible hallucination evaluation framework	last week

输出文件

内容说明

experiment_results.csv	保存四种方法在每一道测试题上的回答结果。
experiment_eval_detail.csv	保存逐题评价明细，包括标准答案、模型回答、检索证据、是否正确、是否幻觉、是否拒答等信息。
experiment_metrics.csv	保存四种方法的汇总指标，包括正确率、幻觉率、覆盖率、失败率和平均时延等。
experiment_report.md	自动生成的实验报告摘要，用于快速查看整体实验结果和方法对比。

该代码结构体现了本文实验的可复现性。首先，测试集和知识库以独立文件形式存放在 data 目录中，便于检查每一道题的来源和证据内容。其次，Base LLM、RAG、RAG + Verify 和 RAG + CoVe 四种方法均通过同一脚本统一调用，避免因手工运行造成流程不一致。再次，评价结果不仅保存汇总指标，也保存逐题判定明细，使读者能够追踪每一个实验结论的来源。最后，仓库中提供 requirements.txt 和 requirements-frozen.txt，便于在相同依赖环境下重新运行实验，降低因环境差异导致结果不一致的可能性。

因此，本文实验不仅给出了最终的统计结果，还保留了从原始数据、模型回答、检索证据、逐题评价到汇总指标的完整链路。这样的设计能够增强课程报告的可信度，

也使实验结论不再只是文字描述，而是可以通过代码和数据进行复查的可复现实验结果。

为验证工程实现是否可靠，本文进一步记录了两个关键故障及修复过程：原评测器使用整句精确匹配，导致正确改写也被判为幻觉；BGE 不可用时原代码退化为随机哈希向量，导致检索结果与语义无关。修复后采用字符 n-gram TF-IDF 检索，并将正确、幻觉、拒答和调用失败分开统计。

表 8 不同实验设置的结果对比

实验设置	幻觉率↓	正确率↑	现象解释
Base LLM	36.0%	64.0%	覆盖率最高，但无证据约束。
RAG	30.5%	57.0%	误检文档会直接引入错误。
RAG+Verify	17.2%	53.0%	通过置信度和候选间隔拒答。
RAG+CoVe	14.3%	12.0%	独立核查阈值过严，覆盖率仅 14%。

5.5 典型样例分析

为进一步说明不同方法在实际问答中的表现，本文从实验结果中选取四类具有代表性的样例进行分析。相比只给出总体正确率、幻觉率和覆盖率，典型样例能够更直观地展示不同方法产生差异的原因，也能够说明检索、验证和拒答机制在幻觉缓解过程中的具体作用。

表 7 典型样例分析

案例类型	问题	标准答案	系统表现	分析
Base LLM 幻觉	标准大气压下水的沸点是多 少？	100 摄氏度	Base LLM 直接回答时给出了不准确的数值或含糊解释；RAG 检索到“标准大气压下水的沸点为 100℃”这一证据后，能够给出正确答	该样例说明，基础大模型虽然能够生成流畅回答，但在具体事实数值上仍可能出现错误。引入检索后，模型回答受到外部证据约束，能够有效减少凭参数记忆生成错

案例类型	问题	标准答案	系统表现	分析
RAG 误检	Python 最新版本是否仍为 3.8?	不是, Python 3.8 已不是最新版本。	案。 RAG 在检索时可能命中过时文档, 并依据“Python 最新版本是 3.8”之类的过时信息作答, 导致生成结果与当前事实不一致。	误事实的情况。 该样例说明, RAG 并不是万能的。检索增强生成的前提是检索到的文档本身可靠且及时; 如果知识库中存在过时信息或事实错误文档, 普通 RAG 仍可能基于错误证据生成幻觉。
Verify 有效拒答	某一概念在知识库中存在多个语义相近但结论不同的解释时, 应如何作答?	应根据明确证据回答; 证据不足时应拒答或提示不确定。	普通 RAG 在检索到相似文档后可能直接生成答案; RAG + Verify 检测到最高相关证据分数不足, 或第一、第二候选证据分数差距较小, 因此选择拒答。	该样例说明, 验证机制能够在证据不足或检索结果存在歧义时阻止系统强行回答。虽然拒答会降低答案覆盖率, 但能够减少无充分证据支持的错误输出。
CoVe 过度保守	数据库主键的作用是什么?	主键用于唯一标识表中的一条记录。	RAG 或 RAG + Verify 可以根据知识库中的相关材料给出正确答案; 但 RAG + CoVe 由于关键词覆盖率或一致性检查未达到阈值, 可能选择拒答。	该样例说明, CoVe 的链式验证策略能够进一步降低幻觉风险, 但如果验证阈值设置过严, 也会把部分本可以回答的问题拦截掉。低幻觉率并不必然代表系统整体效果最好, 还需要结合覆盖率和可用性共同判断。

从上述四类样例可以看出, 不同方法的差异并不只是最终数值上的变化, 而是来源于回答机制的不同。Base LLM 主要依赖模型参数知识, 优点是覆盖率高, 但缺少证据约束, 容易在具体事实、数值和专业概念上产生错误。RAG 通过引入外部知识, 可以降低一部分幻觉, 但当检索到的材料本身错误、过时或与问题只是表面相似时, 仍可能出现误检和错误生成。RAG + Verify 在 RAG 基础上增加了证据充分性判断, 能够在检索结果不可靠时拒答, 从而降低实际作答中的幻觉率。RAG + CoVe 则进一步强化一致性检查, 表现更加保守, 虽然幻觉率最低, 但也会明显降低答案覆盖率。

因此，典型样例与总体统计结果相互印证：检索机制能够改善基础大模型的事实准确性，验证机制能够进一步降低幻觉风险，但过强的验证与拒答策略会牺牲系统可用性。可信问答系统的设计不能只追求最低幻觉率，而应根据具体应用场景在正确率、幻觉率、覆盖率和响应效率之间进行综合权衡。

六、结论与收获

6.1 结论

本文围绕大语言模型幻觉检测与缓解问题展开研究，在梳理近三年相关研究进展的基础上，构建了 Base LLM、RAG、RAG + Verify 和 RAG + CoVe 四类对比实验方案，并基于 100 条事实型问答样本对不同方法的幻觉率、正确率、覆盖率和响应效果进行了综合分析。实验结果表明，单纯依赖大语言模型参数知识进行回答时，系统虽然具有较高的答案覆盖率，但缺乏外部证据约束，容易生成表述流畅但事实依据不足的内容；引入检索增强生成后，模型能够在一定程度上利用外部知识降低幻觉风险；进一步加入验证机制和拒答机制后，系统输出的可靠性得到提升，但覆盖率也随之下降。由此可见，大语言模型幻觉缓解并非单一模型能力问题，而是涉及检索质量、证据验证、拒答策略和评价指标设计的系统性问题。

首先，实验结果表明，RAG 并不能从根本上消除大语言模型幻觉。检索增强生成的优势在于能够将外部知识引入生成过程，减少模型仅依赖参数记忆进行推断的情况。然而，当知识库中存在语义相近的干扰文档、事实错误文档或过时信息时，普通 RAG 仍可能检索到不充分甚至错误的证据，并基于这些证据生成不可靠答案。这说明，检索增强生成只能降低模型凭空编造的概率，并不能保证最终答案一定正确。因此，在可信问答系统中，不能仅以“是否检索到相关文档”作为回答依据，还需要进一步判断检索证据与用户问题之间的相关性、充分性和一致性。

其次，验证机制能够有效降低幻觉率，但会带来答案覆盖率下降的问题。RAG + Verify 和 RAG + CoVe 相较于普通 RAG 增加了证据验证与门控判断环节，当检索分数不足、候选证据差异不明显或关键词覆盖程度较低时，系统倾向于拒绝回答。该策略能够有效减少无证据支持的错误输出，使实际作答中的幻觉率进一步降低。然而，过于严格的验证阈值也可能导致部分原本可以回答的问题被系统拒答，从而降低整体可用性。由此可见，幻觉缓解不应仅追求最低幻觉率，而应在正确率、幻觉率、覆盖率和系统响应效率之间进行综合权衡。对于不同应用场景，验证阈值和拒答策略也应根据风险等级进行动态调整。

再次，不同风险等级的应用场景应采用差异化的幻觉缓解策略。在医学、法律、金融、工程决策和学术引用等高风险场景中，错误回答可能造成较严重后果，因此系统应优先保证可靠性和可追溯性。当证据不足或检索结果存在冲突时，模型应主动拒答或明确提示不确定性，而不应生成缺乏依据的肯定性结论。相比之下，在普通学习问答、日常知识查询和低风险辅助写作场景中，可以采用相对宽松的 RAG + Verify 策略，在保证基本事实一致性的前提下提高答案覆盖率和用户体验。因此，可信大模型系统不应采用单一固定的生成策略，而应根据任务风险、证据质量和用户需求选择合适的回答方式。

综上所述，本文认为，大语言模型幻觉问题的治理应从“模型直接生成”转向“检索—生成—验证—拒答”的闭环式系统架构。该架构首先通过检索模块获取外部证据，随后基于证据生成候选答案，再通过验证模块判断答案是否能够被证据支持，最后在证据不足或存在冲突时执行拒答或不确定性提示。相比单次生成模式，闭环式架构能够在一定程度上提高系统输出的可解释性、可控性和可信性。虽然该方案会增加系统复杂度和计算成本，但对于需要事实准确性和证据支撑的应用场景而言，这种代价是必要且具有实际意义的。

本研究仍存在一定局限性。首先，实验数据规模相对有限，测试集主要围绕事实型问答构造，尚未覆盖长文本生成、多轮对话和复杂推理等更复杂场景。其次，本文采用的检索方式主要基于 TF-IDF 和字符 n-gram，虽然具有较好的可复现性，但在语义

理解能力方面仍弱于向量检索和重排序模型。最后，Verify 和 CoVe 的验证规则仍以启发式阈值为主，对语义层面证据支持关系的判断能力有限。未来研究可以进一步引入向量检索、交叉编码器重排序、自然语言推理模型以及更细粒度的原子事实验证方法，从而提高幻觉检测与缓解系统的准确性和鲁棒性。

通过本文研究可以看出，可信人工智能系统的建设不能仅依赖模型规模提升，也不能只关注单一准确率指标。一个实际可用的大模型系统，应同时具备事实约束能力、证据追溯能力、不确定性表达能力和风险控制能力。对于大语言模型而言，真正的可信性不仅体现在能够回答更多问题，更体现在能够区分“有证据支持的回答”和“证据不足时的不确定”。因此，未来大模型应用的发展方向，应从追求生成能力本身，进一步转向追求可验证、可解释和可控制的可信生成能力。

6.2 团队技术分工与个人收获

表 9 团队技术分工

成员	技术分工	完成情况
叶浚宇（组长）	负责方案策划与整体统筹，包括选题确定、近三年文献调研、技术路线设计、评价指标制定、实验方案规划、结果分析、图表绘制以及论文撰写与汇报内容统筹。	完成
王晨枫（组员）	负责具体代码实现与工程复现，包括 Base LLM、RAG、Verify 和 CoVe 模块开发，模型 API 与本地 Ollama 接入，检索及评测逻辑调试、实验运行、故障修复和结果文件整理。	完成

通过本次报告，我对大语言模型的能力边界有了更清晰的认识：大模型并不是一个天然可靠的知识库，它更像一个强大的语言生成器。要把大模型用于真实场景，必须加入检索、证据、验证和拒答机制。这也让我认识到，人工智能系统的价值不仅来自模型规模，还来自系统设计、数据质量和可靠性评估。

参考文献

- [1] LIN S, HILTON J, EVANS O. TruthfulQA: Measuring How Models Mimic Human Falsehoods[C]//Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics. Dublin: Association for Computational Linguistics, 2022: 3214–3252. DOI:10.18653/v1/2022.acl-long.229.
- [2] MANAKUL P, LIUSIE A, GALES M. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models[C]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Singapore: Association for Computational Linguistics, 2023: 9004–9017. DOI:10.18653/v1/2023.emnlp-main.557.
- [3] LI J, CHENG X, ZHAO W X, et al. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models[C]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Singapore: Association for Computational Linguistics, 2023: 6449–6464. DOI:10.18653/v1/2023.emnlp-main.397.
- [4] MIN S, KRISHNA K, LYU X, et al. FActScore: Fine-Grained Atomic Evaluation of Factual Precision in Long Form Text Generation[C]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Singapore: Association for Computational Linguistics, 2023: 12076–12100. DOI:10.18653/v1/2023.emnlp-main.741.
- [5] ES S, JAMES J, ESPINOSA ANKE L, et al. RAGAs: Automated Evaluation of Retrieval Augmented Generation[C]//Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations. St. Julians: Association for Computational Linguistics, 2024: 150–158. DOI:10.18653/v1/2024.eacl-demo.16.
- [6] DHULIAWALA S, KOMEILI M, XU J, et al. Chain-of-Verification Reduces Hallucination in Large Language Models[C]//Findings of the Association for Computational Linguistics: ACL 2024. Bangkok: Association for Computational Linguistics, 2024: 3563–3578. DOI:10.18653/v1/2024.findings-acl.212.
- [7] VU T, IYYER M, WANG X, et al. FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation[C]//Findings of the Association for Computational Linguistics: ACL 2024. Bangkok: Association for Computational Linguistics, 2024: 13697–13720. DOI:10.18653/v1/2024.findings-acl.813.
- [8] GAO Y, XIONG Y, GAO X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[EB/OL]. arXiv:2312.10997, 2023[2026-06-22]. <https://arxiv.org/abs/2312.10997>.
- [9] HUANG L, YU W, MA W, et al. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions[J]. ACM Transactions on Information Systems, 2025, 43(2): 42:1–42:55. DOI:10.1145/3703155.

[10] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval–Augmented Generation for Knowledge–Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems 33. Red Hook: Curran Associates, 2020: 9459–9474.

附录代码

```
//Base_llm

import time

from src.config import get_llm

class BasePipeline:

    def __init__(self):

        self.llm = get_llm()

    def run(self, question: str) -> dict:

        start_time = time.time()

        try:

            answer = self.llm.invoke(

                f"请直接、简洁地回答问题。不要编造不确定的信息。\\n 问题: {question}"

            ).content

        except Exception as error:

            answer = f"【降级回答】模型调用失败: {type(error).__name__}"

        return {

            "answer": answer,

            "latency": time.time() - start_time,

            "context": [],

        }

//rag

import time
```

```

from src.config import get_llm

class BasePipeline:

    def __init__(self):

        self.llm = get_llm()

    def run(self, question: str) -> dict:

        start_time = time.time()

        try:

            answer = self.llm.invoke(

                f"请直接、简洁地回答问题。不要编造不确定的信息。\\n 问题: {question}"

            ).content

        except Exception as error:

            answer = f"【降级回答】模型调用失败: {type(error).__name__}"

        return {

            "answer": answer,

            "latency": time.time() - start_time,

            "context": [],

        }

//rag+cove

import os

import re

import time

from src.pipelines.rag_baseline import RAGPipeline

class RAGCovePipeline(RAGPipeline):

```

```

""""Retrieval plus independent keyword checks and conservative refusal."""
def __init__(self):
    super().__init__()

    self.cove_score = float(os.getenv("COVE_MIN_SCORE", "0.16"))

    self.keyword_coverage = float(os.getenv("COVE_KEYWORD_COVERAGE",
"0.45"))

    @staticmethod
    def _keywords(question: str) -> list[str]:
        chunks = re.findall(r"[\u4e00-\u9fff]{2,}[A-Za-z0-9.+/-]+", question)

        stop = {"什么", "哪个", "哪些", "多少", "是否", "通常", "主要", "为什么"}

        return [chunk for chunk in chunks if chunk not in stop]

    def run(self, question: str) -> dict:
        start_time = time.time()

        docs = self.retriever.retrieve(question, top_k=5)

        top = docs[0]

        evidence = f"{top.metadata.get('title', '')}{top.page_content}"

        keywords = self._keywords(question)

        coverage = (
            sum(keyword in evidence for keyword in keywords) / len(keywords)
            if keywords
            else 0.0
        )

        if (
            top.metadata.get("score", 0.0) >= self.cove_score

```

```

        and coverage >= self.keyword_coverage

    ):

        answer = top.page_content

    else:

        answer = "【拒答】独立核查未找到足够一致的证据，无法可靠回答。"

    return {

        "answer": answer,

        "latency": time.time() - start_time,

        "context": [doc.page_content for doc in docs],

    }

```

//rag+verify

```
import os
```

```
import time
```

```
from src.pipelines.rag_baseline import RAGPipeline
```

```
class RAGVerifyPipeline(RAGPipeline):
```

```
    """RAG with confidence and ambiguity checks before answering."""
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.verify_score = float(os.getenv("VERIFY_MIN_SCORE", "0.18"))
```

```
        self.verify_margin = float(os.getenv("VERIFY_MIN_MARGIN", "0.035"))
```

```
    def run(self, question: str) -> dict:
```

```
        start_time = time.time()
```

```
        docs = self.retriever.retrieve(question, top_k=3)
```

```

top_score = docs[0].metadata.get("score", 0.0)

second_score = docs[1].metadata.get("score", 0.0) if len(docs) > 1 else 0.0

if top_score >= self.verify_score and top_score - second_score >=
self.verify_margin:

    answer = docs[0].page_content

else:

    answer = "【拒答】检索证据的相关性或唯一性不足，无法可靠回答。"

return {

    "answer": answer,

    "latency": time.time() - start_time,

    "context": [doc.page_content for doc in docs],

}

```